Lab Manual

Programming for AI
AI-212

By

Haris Awan **2023-BSAI-023**

Submitted to

Mr Samraiz

Bachelor of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCES

LAB 1

Q1. Banking Transaction Validator with PIN

Simulate a bank ATM.
User enters a 4-digit PIN (check if valid).
Display account balance and ask for withdrawal amount.
If valid → deduct + update balance.
If invalid attempts ≥ 3 → block card.
Use operators to compute service charges (2% per withdrawal).

1.py

lab 1 > 1.py > ...

Windsurf: Refactor | Explain | Generate Docstring |

```python
def atm_simulator():
    pin = "1234"
    balance = 1000
    attempts = 0
    while attempts < 3:
        entered = input("Enter 4-digit PIN: ")
        if entered == pin:
            print("PIN Accepted ")
            print(f"Your Balance: {balance}")
            try:
                amount = int(input("Enter withdrawal amount: "))
                charges = amount * 0.02
                total = amount + charges
                if total <= balance:
                    balance -= total
                    print(f"Withdrawn: {amount}, Service Charges: {charges}, New Balance: {balance}")
                else:
                    print("Insufficient Balance ")
            except ValueError:
                print("Invalid amount entered ")
            break
        else:
            attempts += 1
            print("Invalid PIN ")
    if attempts == 3:
        print("Card Blocked ")
atm_simulator()
```

output:

```
Enter 4-digit PIN: 4567
Invalid PIN
Enter 4-digit PIN: 1234
PIN Accepted
Your Balance: 1000
Enter withdrawal amount: 1200
Insufficient Balance
```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.
If stock available, deduct from inventory.
Apply discount slabs (10%, 15%, 20%).
Print final bill with tax (5%).

```python
def grocery_store():
    items = {
        "eggs": {"stock": 10, "price": 550},
        "milk": {"stock": 20, "price": 170},
        "bread": {"stock": 15, "price": 200}
    }
    bill = 0
    while True:
        print("\nAvailable items:")
        for item, info in items.items():
            print(f"{item.capitalize()} - Rs.{info['price']} (Stock: {info['stock']})")
        choice = input("\nEnter item name (or 'done' to finish): ").lower()
        if choice == "done":
            break

        if choice in items:
            try:
                qty = int(input("Enter quantity: "))
                if qty <= 0:
                    print("Quantity must be positive ")
                    continue
                if items[choice]["stock"] >= qty:
                    cost = items[choice]["price"] * qty
                    items[choice]["stock"] -= qty
                    bill += cost
                    print(f"Added {qty} {choice}(s) → Rs.{cost}")
                else:
                    print("Insufficient stock ")
            except ValueError:
                print("Invalid quantity ")
        else:
            print("Item not available ")
```

```python
def grocery_store():
    if bill > 500:
        discount = 0.2
    elif bill > 300:
        discount = 0.15
    elif bill > 100:
        discount = 0.1
    else:
        discount = 0
    bill -= bill * discount
    tax = bill * 0.05
    final_bill = bill + tax
    print(f"\nFinal Bill: Rs.{final_bill:.2f} (including 5% tax)")
grocery_store()
```

output:

```
Available items:
Eggs - Rs.550 (Stock: 10)
Milk - Rs.170 (Stock: 20)
Bread - Rs.200 (Stock: 15)

Enter item name (or 'done' to finish): bread
Enter quantity: 2
Added 2 bread(s) → Rs.400

Available items:
Eggs - Rs.550 (Stock: 10)
Milk - Rs.170 (Stock: 20)
Bread - Rs.200 (Stock: 13)

Enter item name (or 'done' to finish): done

Final Bill: Rs.357.00 (including 5% tax)
```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).

Compute weighted average.

Assign grade (A/B/C/D/Fail).

Also, display whether the student is eligible for scholarship (≥80% weighted).

```python
def student_grading():
    weights = (0.1, 0.2, 0.25, 0.2, 0.25)
    marks = []
    for i in range(5):
        marks.append(int(input(f"Enter marks for subject {i+1}: ")))
    weighted = sum(m * w for m, w in zip(marks, weights))
    print(f"Weighted % = {weighted}")
    if weighted >= 80:
        grade, scholarship = "A", "Eligible"
    elif weighted >= 70:
        grade, scholarship = "B", "Not Eligible"
    elif weighted >= 60:
        grade, scholarship = "C", "Not Eligible"
    elif weighted >= 50:
        grade, scholarship = "D", "Not Eligible"
    else:
        grade, scholarship = "Fail", "Not Eligible"
    print(f"Grade: {grade}, Scholarship: {scholarship}")
student_grading()
```

output:

```
Enter marks for subject 1: 70
Enter marks for subject 2: 80
Enter marks for subject 3: 90
Enter marks for subject 4: 100
Enter marks for subject 5: 30
Weighted % = 73.0
Grade: B, Scholarship: Not Eligible
```

Q4. BMI + Health Recommendation

Extend BMI program:

Take weight, height, and age.

Compute BMI.

Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

```python
def bmi_health():
    w = float(input("Enter weight(kg): "))
    h = float(input("Enter height(m): "))
    age = int(input("Enter age: "))
    bmi = w / (h**2)
    print(f"BMI = {bmi:.2f}")
    if bmi < 18.5:
        print("Underweight. Eat protein & exercise.")
    elif bmi < 25:
        print("Normal. Maintain balanced diet.")
    elif bmi < 30:
        print("Overweight. Exercise daily.")
    else:
        print("Obese. Follow strict diet + workout.")
    if age > 50 and bmi > 30:
        print(" Special Care Needed.")
bmi_health()
```

output:

```
Enter weight(kg): 70
Enter height(m): 2
Enter age: 20
BMI = 17.50
Underweight. Eat protein & exercise.
```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.
User selects usage (calls, SMS, data).
Deduct accordingly from balance.
If balance < 20% of initial → print recharge reminder.
Apply bonus balance if recharge > 1000.

```python
def telecom():
    rates = {"call": 1, "sms": 0.5, "data": 2}  # per min, per sms, per MB
    balance = 200
    init_balance = balance
    call = int(input("Minutes used: ")) * rates["call"]
    sms = int(input("SMS used: ")) * rates["sms"]
    data = int(input("MB used: ")) * rates["data"]
    total = call + sms + data
    balance -= total
    print(f"Remaining Balance: {balance}")
    if balance < 0.2 * init_balance:
        print("Recharge Reminder ")
    recharge = int(input("Enter recharge amount: "))
    balance += recharge
    if recharge > 1000:
        balance += 100
        print(" Bonus 100 added")
    print(f"Final Balance: {balance}")
telecom()
```

output:

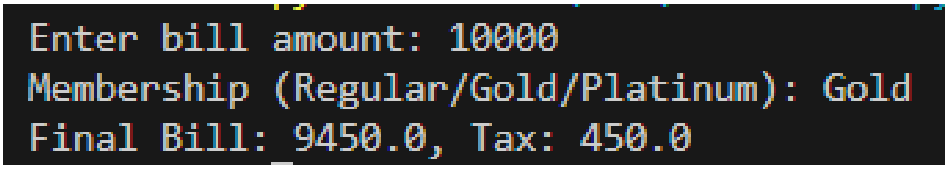
```
Minutes used: 10
SMS used: 100
MB used: 2
Remaining Balance: 136.0
Enter recharge amount: 100
Final Balance: 236.0
```

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).
Apply different discount rules (Gold +10%, Platinum +15%).
If bill > 10000, apply additional 5% flat.
Show itemized receipt with tax breakdown.

```python
def ecommerce():
    bill = float(input("Enter bill amount: "))
    member = input("Membership (Regular/Gold/Platinum): ").lower()
    if member == "gold":
        bill -= bill * 0.1
    elif member == "platinum":
        bill -= bill * 0.15
    if bill > 10000:
        bill -= bill * 0.05
    tax = bill * 0.05
    print(f"Final Bill: {bill+tax}, Tax: {tax}")
    return
ecommerce()
```

output:



Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.
Classify into categories: Stable, Observation, Urgent, Critical.
If critical → suggest "Admit in ICU."
If urgent → suggest "Immediate doctor attention."
Use nested conditions.

```python
def hospital():
    t = float(input("Temp: "))
    p = int(input("Pulse: "))
    o2 = int(input("Oxygen: "))
    bp = int(input("BP: "))
    if o2 < 85 or bp < 80:
        print("Critical → Admit in ICU")
    elif o2 < 92 or bp < 90:
        print("Urgent → Immediate doctor attention")
    elif t > 100 or p > 100:
        print("Observation needed")
    else:
        print("Stable")

hospital()
```

output:

```
Temp: 100
Pulse: 77
Oxygen: 80
BP: 120
Critical → Admit in ICU
```

Q8. Online Examination Grader with Negative Marking

Take total marks, obtained marks, and number of wrong answers.
Deduct 0.25 per wrong answer.
Compute percentage.
Assign division + eligibility for scholarship.
If Fail → suggest "Repeat Exam."

```python
def exam():
    total = int(input("Total Marks: "))
    obtained = int(input("Obtained: "))
    wrong = int(input("Wrong Answers: "))
    obtained -= wrong * 0.25
    percent = (obtained / total) * 100
    if percent >= 80:
        div, scholar = "1st Division", "Scholarship Eligible"
    elif percent >= 60:
        div, scholar = "2nd Division", "Not Eligible"
    elif percent >= 40:
        div, scholar = "3rd Division", "Not Eligible"
    else:
        print("Fail → Repeat Exam")
        return
    print(f"Division: {div}, Scholarship: {scholar}")
exam()
```

output:

```
Total Marks: 150
Obtained: 97
Wrong Answers: 53
Division: 3rd Division, Scholarship: Not Eligible
```

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).
Apply child restrictions (No late-night for <12).
Apply senior citizen discount (30%).
Apply peak hour charges (10%).
Print final ticket price.

```python
def movie_ticket():
    base = {"normal": 300, "3d": 500, "imax": 800}
    age = int(input("Age: "))
    show = int(input("Showtime(24hr): "))
    ttype = input("Type (Normal/3D/IMAX): ").lower()
    price = base[ttype]
    if age < 12 and show > 20:
        print("Children not allowed at late night")
        return
    if age > 60:
        price -= price * 0.3
    if 18 <= show <= 22:
        price += price * 0.1
    print(f"Ticket Price: {price}")

movie_ticket()
```

output:

```
Age: 10
Showtime(24hr): 2
Type (Normal/3D/IMAX): Normal
Ticket Price: 300
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend “light formal suit.”

```python
def outfit():
    temp = int(input("Temperature: "))
    weather = input("Weather (Rainy/Sunny/Cold): ").lower()
    event = input("Event (Casual/Business/Party): ").lower()
    if weather == "rainy":
        print("Take an umbrella ")
    if weather == "cold":
        print("Wear gloves ")
    if event == "business" and temp > 35:
        print("Wear light formal suit")
    else:
        print("Dress comfortably for", event)

outfit()
```

output:

```
Temperature: 40
Weather (Rainy/Sunny/Cold): sunny
Event (Casual/Business/Party): party
Dress comfortably for party
```

Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.

Compute minimum number of notes (1000, 500, 100, 50, 10, 1).

Also compute service charge = 0.5%.

Print receipt with remaining balance after deduction.

```python
def atm_withdraw():
    balance = 10000
    amount = int(input("Enter withdrawal: "))
    if amount > balance:
        print("Insufficient Funds")
        return
    notes = {}
    for n in [1000, 500, 100, 50, 10, 1]:
        notes[n], amount = divmod(amount, n)
    charge = amount * 0.005
    balance -= (amount + charge)
    print("Notes:", notes)
    print(f"Remaining Balance: {balance}")
atm_withdraw()
```

output:

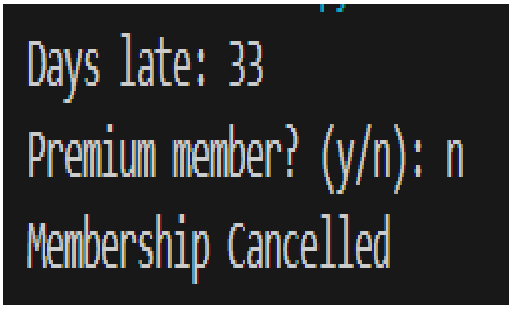
```
Enter withdrawal: 500
Notes: {1000: 0, 500: 1, 100: 0, 50: 0, 10: 0, 1: 0}
Remaining Balance: 10000.0
```

Q12. Library Fine Calculator with Membership Levels

Input days late.
Apply fines (10, 20, 50/day).
If days > 30 → "Membership Cancelled."
If user is Premium member, reduce fine by 50%.
Print detailed fine slip.

```python
def library():
    days = int(input("Days late: "))
    member = input("Premium member? (y/n): ")
    if days > 30:
        print("Membership Cancelled")
        return
    fine = 0
    if days <= 10:
        fine = days * 10
    elif days <= 20:
        fine = days * 20
    else:
        fine = days * 50
    if member == "y":
        fine /= 2
    print(f"Fine: {fine}")

library()
```

output:



Q13. Sales Analysis Dashboard

Store monthly sales in a list (12 entries).

Print max, min, average.

Print months above average.

Use loop + condition to print “Growth” or “Decline” compared to last month.

```python
def sales():
    sales = [int(input(f"Enter sales for month {i+1}: ")) for i in range(12)]
    avg = sum(sales)/12
    print("Max:", max(sales), "Min:", min(sales), "Avg:", avg)
    for i, val in enumerate(sales):
        if val > avg:
            print(f"Month {i+1} above average")
        if i > 0:
            if val > sales[i-1]:
                print(f"Month {i+1}: Growth")
            else:
                print(f"Month {i+1}: Decline")
sales()
```

output:

```
Enter sales for month 1: 12345
Enter sales for month 2: 34567
Enter sales for month 3: 45678
Enter sales for month 4: 56789
Enter sales for month 5: 12345678
Enter sales for month 6: 876543
Enter sales for month 7: 98765
Enter sales for month 8: 567
Enter sales for month 9: 98765
Enter sales for month 10: 5678
Enter sales for month 11: 87654
Enter sales for month 12: 4567
Max: 12345678 Min: 567 Avg: 1138966.3333333333
Month 2: Growth
Month 3: Growth
Month 4: Growth
Month 5 above average
Month 5: Growth
Month 6: Decline
Month 7: Decline
Month 8: Decline
Month 9: Growth
Month 10: Decline
Month 11: Growth
Month 12: Decline
```

Q14. Multiplication Puzzle with Random Blanks

Generate a multiplication table (1–10).
Randomly replace 3 results with "??".

Ask user to guess values.
Track score.
Give performance remark (Excellent/Good/Try Again).

```python
import random
def puzzle():
    score = 0
    table = {i: i*5 for i in range(1,11)}
    blanks = random.sample(list(table.keys()),3)
    for k in blanks:
        ans = int(input(f"5 x {k} = ?? "))
        if ans == table[k]:
            score += 1
    print("Score:", score)
    if score == 3: print("Excellent")
    elif score == 2: print("Good")
    else: print("Try Again")

puzzle()
```

output:

```
5 x 2 = ?? 14
5 x 1 = ?? 6
5 x 5 = ?? 25
Score: 1
Try Again
```

Q15. Password Strength + Re-entry Attempts
Check password rules (length, digit, upper, lower, special).
If weak → ask user to re-enter.
Allow max 3 attempts.
If still weak → "Account Locked."

```python
import re
def password_check():
    attempts = 0
    while attempts < 3:
        pwd = input("Enter password: ")
        if (len(pwd)>=8 and re.search("[A-Z]",pwd) and re.search("[a-z]",pwd)
            and re.search("[0-9]",pwd) and re.search("[^A-Za-z0-9]",pwd)):
            print("Strong Password ")
            return
        else:
            print("Weak Password ")
            attempts += 1
    print("Account Locked ")

password_check()
```

output:

```
Enter password: 12345678
Weak Password
Enter password: h@r1234$
Weak Password
Enter password: H@r1$4567@
Strong Password
```

Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):
Apply slab rates (5/7/10 Rs).
If late_payment → add 15% penalty.
Return bill breakdown.

```python
def calculate_bill(units, late=False):
    if units <= 100:
        amt = units*5
    elif units <= 200:
        amt = 100*5 + (units-100)*7
    else:
        amt = 100*5 + 100*7 + (units-200)*10
    if late:
        amt += amt*0.15
    print(f"Bill = {amt}")
calculate_bill(1200, True)
```

output:

```
Bill = 12880.0
```

Q17. Cricket Match Predictor with Multiple Conditions

Function predict_score(runs, balls, wickets, overs_left):

Compute strike rate & run rate.

If run rate < 5 and wickets > 6 → "Losing."

If run rate > 7 and wickets < 5 → "Winning."

Else → "Balanced."

```python
def predict_score(runs, balls, wickets, overs_left):
    run_rate = runs / (20-overs_left)
    if run_rate < 5 and wickets > 6:
        print("Losing")
    elif run_rate > 7 and wickets < 5:
        print("Winning")
    else:
        print("Balanced")
predict_score(int(input("Runs: ")), int(input("Balls: ")), int(input("Wickets: ")), int(input("Overs left: ")))
```

output:

```
Runs: 147
Balls: 120
Wickets: 4
Overs left: 30
Balanced
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):
Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.
If days > 7 → 20% discount.
If member → extra 10% discount.
Add 12% tax.
Return bill slip.

```
Windsurf: Refactor | Explain | Generate Docstring |
def book_room(type, days, is_member):
    price = {"standard":2000,"deluxe":4000,"suite":6000}[type]
    cost = price*days
    if days>7: cost*=0.8
    if is_member: cost*=0.9
    tax = cost*0.12
    print(f"Final Bill: {cost+tax}")
book_room(input("Room Type (standard/deluxe/suite): ").lower(), int(input("Number of days: ")), input("Loyalty member? (y/n):
```

output:

```
Room Type (standard/deluxe/suite): suite
Number of days: 23
Loyalty member? (y/n): n
Final Bill: 123648.0
```

Q19. Flight Fare Calculator with International Tax

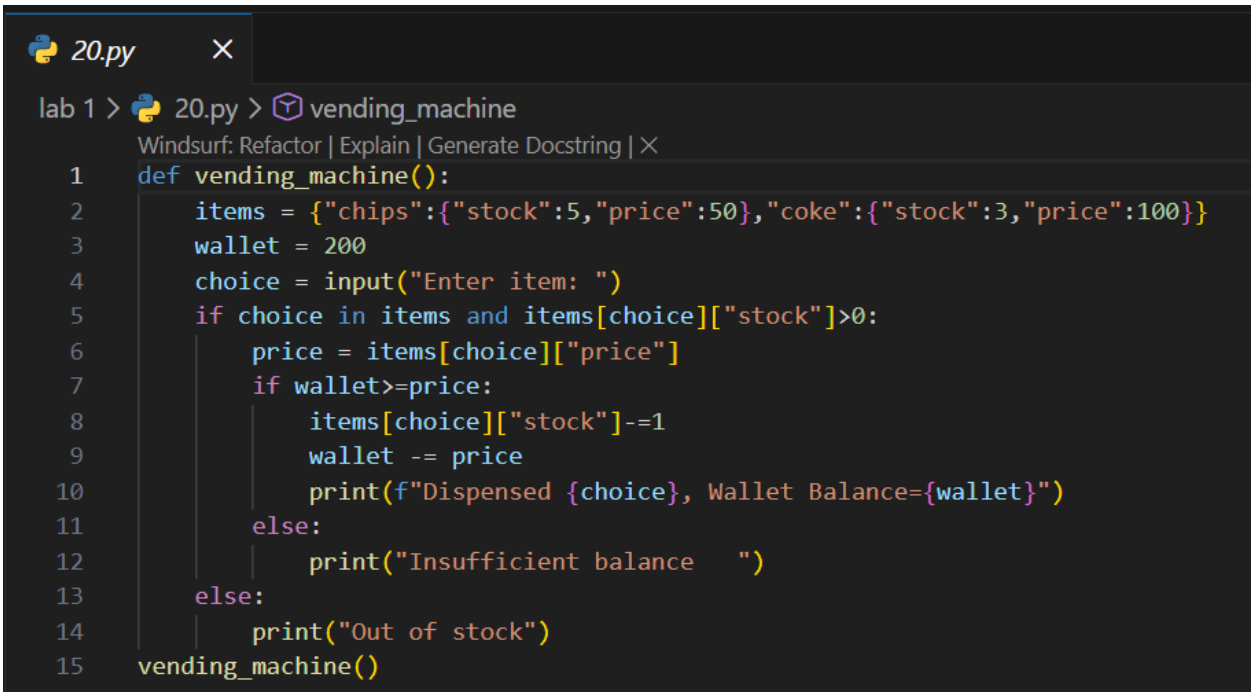
Function calculate_fare(distance, class_type, international=False):
Economy: 10/unit, Business: 25/unit, First: 40/unit.
Add fuel surcharge 500.
If international → add 18% tax.
If booking in advance (>30 days) → 10% discount.

output:

```
Distance (km): 20
Class (economy/business/first): first
International? (y/n): y
Advance booking? (y/n): y
Fare: 1380.6000000000001
```

Q20. Smart Vending Machine with Wallet System

Function vending_machine(item, amount_paid, wallet_balance):
Items stored in dict with stock + price.
If sufficient → dispense + return change.
Update wallet balance.
If stock = 0 → suggest alternative item.



output:

```
Enter item: chips
Dispensed chips, Wallet Balance=150
```

LAB # 2 (OOP in Python)

Question # 1: Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

1.py

lab 2 > 1.py > BankAccount > _init_

```
Windsurf: Refactor | Explain
class InsufficientFunds(Exception):
    pass
Windsurf: Refactor | Explain
class BankAccount:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def _init_(self, account_number, holder_name, balance=0.0):
        self.account_number = account_number
        self.holder_name = holder_name
        self.balance = balance
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}. New Balance = {self.balance}")
        else:
            print("Deposit amount must be positive.")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount.")
        elif amount > self.balance:
            print("Error: Insufficient balance.")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. Remaining Balance = {self.balance}")

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def display_balance(self):
        print(f"Account {self.account_number} - {self.holder_name}: Balance = {self.balance}")

```

output:

```
Deposited 200. New Balance = 700
Withdrew 100. Remaining Balance = 600
Account 101 - Haris: Balance = 600
Deposited 200. New Balance = 700
Withdrew 100. Remaining Balance = 600
Account 101 - Haris: Balance = 600
Deposited 200. New Balance = 700
Withdrew 100. Remaining Balance = 600
Account 101 - Haris: Balance = 600
```

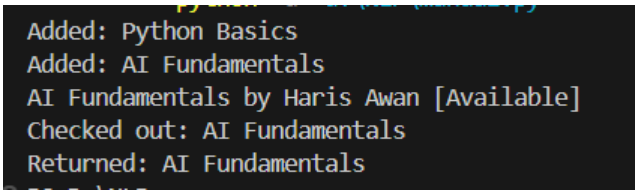
Question # 2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

2.py

lab 2 > 2.py > Library > checkout

```python
Windsurf: Refactor | Explain
class Book:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def _init_(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def _str_(self):
        status = "Available" if self.available else "Checked Out"
        return f"{self.title} by {self.author} [{status}]"
Windsurf: Refactor | Explain
class Library:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def _init_(self):
        self.books = []
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_book(self, book):
        self.books.append(book)
        print(f"Added: {book.title}")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def search_by_title(self, title):
        return [b for b in self.books if title.lower() in b.title.lower()]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def search_by_author(self, author):
        return [b for b in self.books if author.lower() in b.author.lower()]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def checkout(self, book):
        if book.available:
            book.available = False
            print(f"Checked out: {book.title}")
        else:
            print("Book not available!")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def return_book(self, book):
        book.available = True
        print(f"Returned: {book.title}")
```

output:



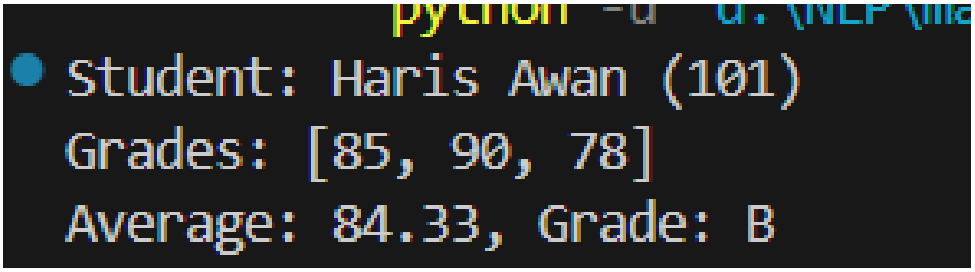
Question # 3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

3.py

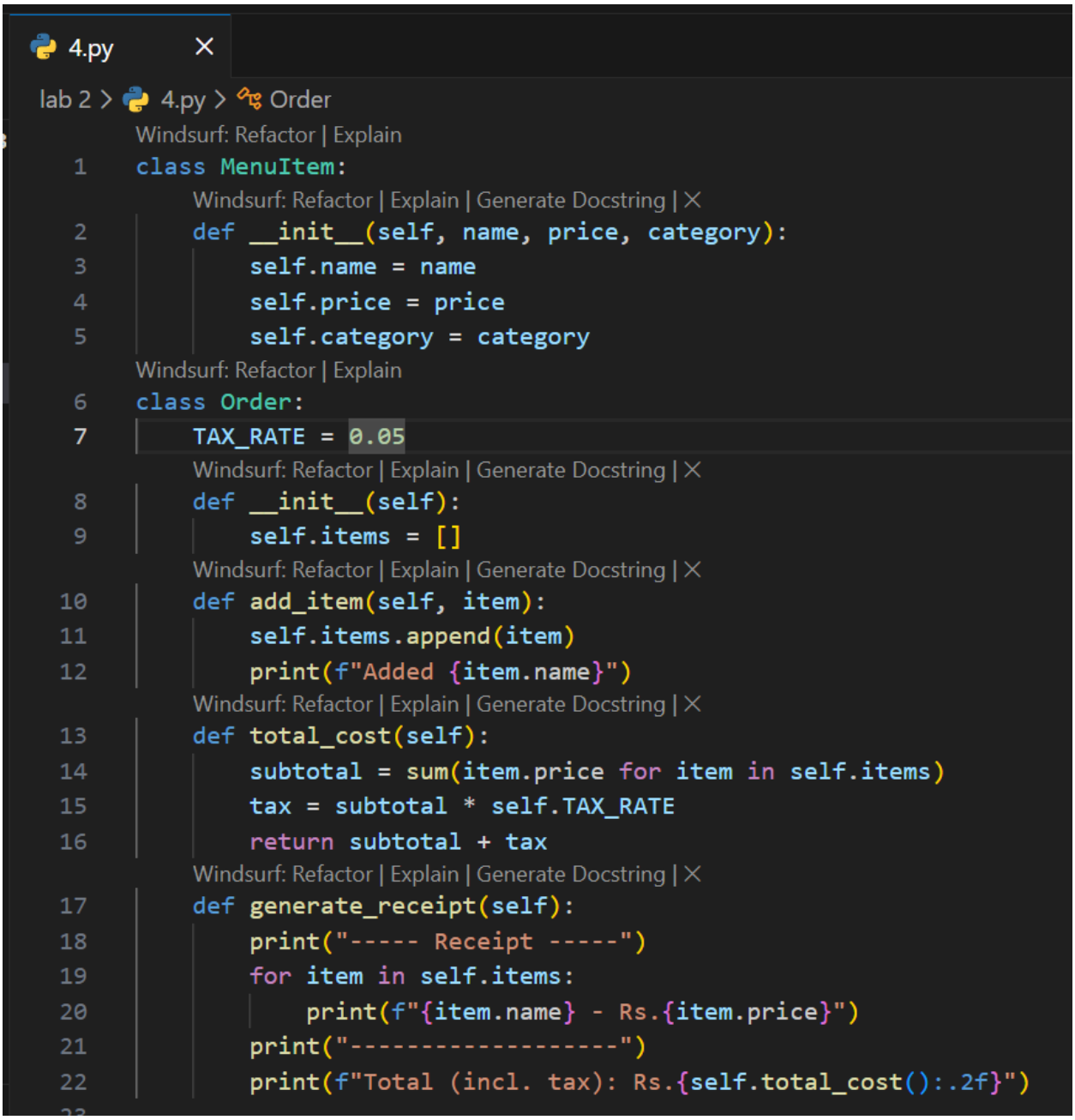
lab 2 > 3.py > Student

```python
Windsurf: Refactor | Explain
class Student:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_grade(self, grade):
        self.grades.append(grade)

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def average(self):
        return sum(self.grades) / len(self.grades) if self.grades else 0

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def letter_grade(self):
        avg = self.average()
        if avg >= 90: return "A"
        elif avg >= 80: return "B"
        elif avg >= 70: return "C"
        elif avg >= 60: return "D"
        else: return "F"

    Windsurf: Refactor | Explain | Generate Docstring | ×
    def display_info(self):
        print(f"Student: {self.name} ({self.student_id})")
        print(f"Grades: {self.grades}")
        print(f"Average: {self.average():.2f}, Grade: {self.letter_grade()}")

# Example use:
student1 = Student(101, "Haris Awan")
student1.add_grade(85)
student1.add_grade(90)
student1.add_grade(78)
student1.display_info()
```

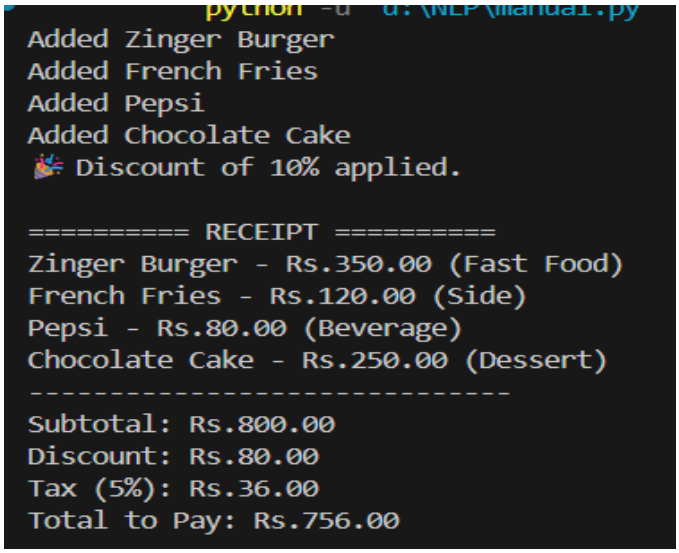
output:



Question # 4: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.



output:



Question # 5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

5.py

lab 2 > 5.py > RentalSystem > __init__

```python
class Vehicle:
    def __init__(self, plate, model, price_per_day):
        self.plate = plate
        self.model = model
        self.price_per_day = price_per_day
        self.available = True
Windsurf: Refactor | Explain
class Car(Vehicle):
    pass
Windsurf: Refactor | Explain
class Motorcycle(Vehicle):
    pass
Windsurf: Refactor | Explain
class Truck(Vehicle):
    pass
Windsurf: Refactor | Explain
class RentalSystem:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.vehicles = []
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_vehicle(self, v):
        self.vehicles.append(v)
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def rent_vehicle(self, plate, days):
        for v in self.vehicles:
            if v.plate == plate and v.available:
                v.available = False
                cost = v.price_per_day * days
                print(f"Rented {v.model} for {days} days. Cost = Rs.{cost}")
                return cost
        print("Vehicle not available!")
rental = RentalSystem()
car1 = Car("ABC-123", "Toyota Corolla", 5000)
bike1 = Motorcycle("XYZ-456", "Honda 125", 1500)
truck1 = Truck("TRK-789", "Hino Truck", 8000)
rental.add_vehicle(car1)
rental.rent_vehicle("ABC-123", 3)
```

output:

```
Rented Toyota Corolla for 3 days. Cost = Rs.15000
```

Question # 6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary,

annual salary, and apply raises. Include functionality to track employee details and employment duration.

6.py

lab 2 > 6.py > Employee

```python
Windsurf: Refactor | Explain
class Employee:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, emp_id, name, position, salary):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def monthly_salary(self):
        return self.salary
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def annual_salary(self):
        return self.salary * 12
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def apply_raise(self, percent):
        self.salary += self.salary * percent / 100
        print(f"{self.name}'s new salary: {self.salary}")
emp1 = Employee(101, "Haris", "Software Engineer", 80000)
print("Monthly Salary:", emp1.monthly_salary())
print("Annual Salary:", emp1.annual_salary())
emp1.apply_raise(10)
```

output:

```
Monthly Salary: 80000
Annual Salary: 960000
Haris's new salary: 88000.0
```

Question # 7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

7.py

lab 2 > 7.py > Product

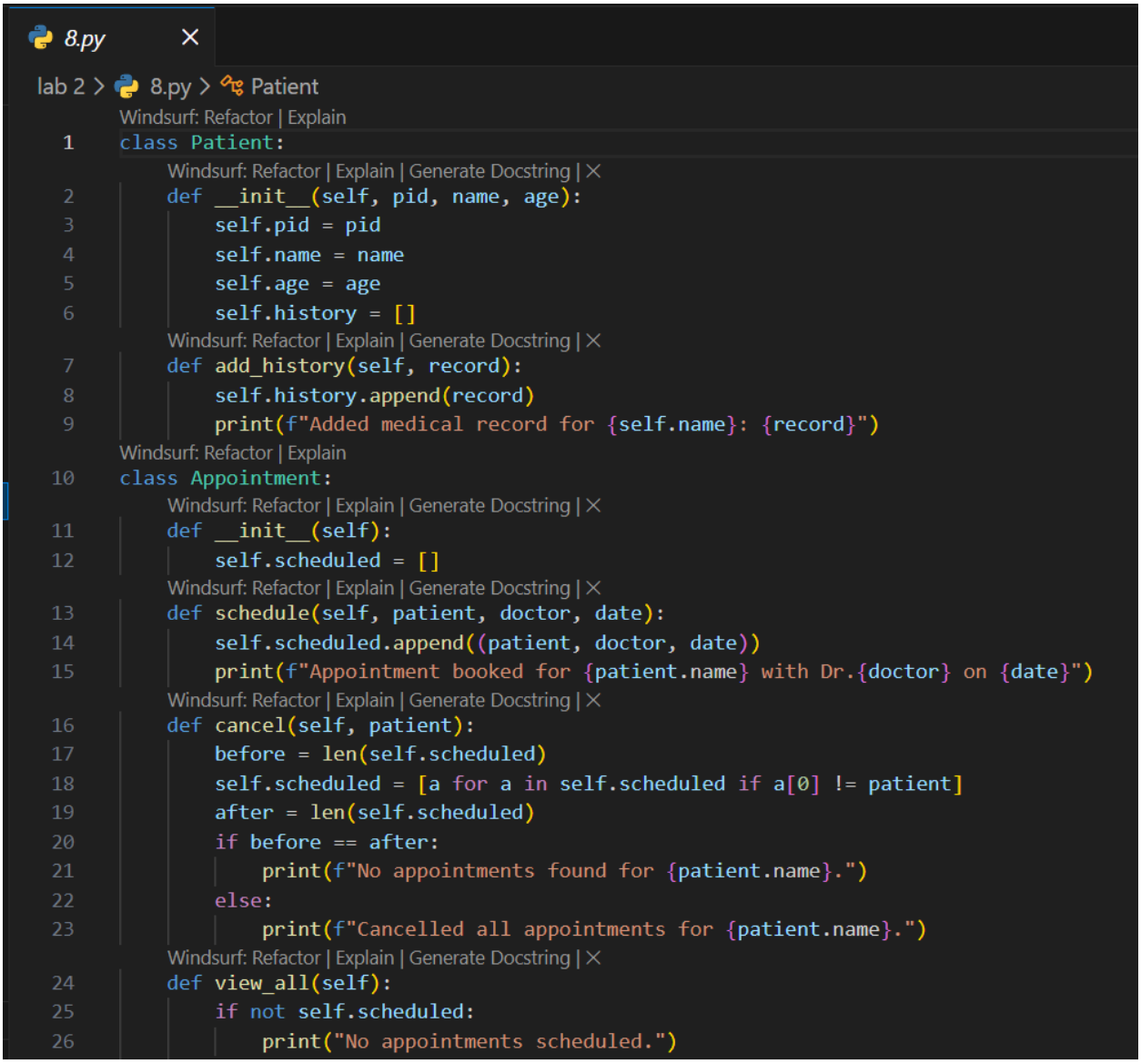
```python
Windsurf: Refactor | Explain
class Product:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, pid, name, price, stock):
        self.pid = pid
        self.name = name
        self.price = price
        self.stock = stock
Windsurf: Refactor | Explain
class ShoppingCart:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.items = []
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_product(self, product, qty):
        if product.stock >= qty:
            self.items.append((product, qty))
            product.stock -= qty
            print(f"Added {qty} x {product.name} to cart.")
        else:
            print("Not enough stock!")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def remove_product(self, product):
        for item in self.items:
            if item[0] == product:
                self.items.remove(item)
                product.stock += item[1]
                print(f"Removed {product.name} from cart.")
                return
        print("Product not found in cart!")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def total(self):
        return sum(p.price * q for p, q in self.items)
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def checkout(self):
        total_cost = self.total()
        print(f"Checkout complete. Total cost: Rs.{total_cost}")
        self.items.clear()
```

```
    p1 = Product(1, "Laptop", 120000, 5)
    p2 = Product(2, "Headphones", 3000, 10)
    p3 = Product(3, "Mouse", 1500, 15)
    cart = ShoppingCart()
    cart.add_product(p1, 1)
    cart.add_product(p2, 2)
    print("Total:", cart.total())
    cart.remove_product(p2)
    print("Total after removing:", cart.total())
    cart.checkout()
```

output:

```
Added 1 x Laptop to cart.
Added 2 x Headphones to cart.
Total: 126000
Removed Headphones from cart.
Total after removing: 120000
Checkout complete. Total cost: Rs.120000
```

Question # 8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.



8.py

lab 2 > 8.py > Patient

```python
Windsurf: Refactor | Explain
class Patient:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, pid, name, age):
        self.pid = pid
        self.name = name
        self.age = age
        self.history = []
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_history(self, record):
        self.history.append(record)
        print(f"Added medical record for {self.name}: {record}")
Windsurf: Refactor | Explain
class Appointment:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.scheduled = []
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def schedule(self, patient, doctor, date):
        self.scheduled.append((patient, doctor, date))
        print(f"Appointment booked for {patient.name} with Dr.{doctor} on {date}")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def cancel(self, patient):
        before = len(self.scheduled)
        self.scheduled = [a for a in self.scheduled if a[0] != patient]
        after = len(self.scheduled)
        if before == after:
            print(f"No appointments found for {patient.name}.")
        else:
            print(f"Cancelled all appointments for {patient.name}.")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def view_all(self):
        if not self.scheduled:
            print("No appointments scheduled.")
```

```
            else:
                print("\n--- Appointment List ---")
                for p, d, dt in self.scheduled:
                    print(f"{p.name} → Dr.{d} on {dt}")
# Example usage
p1 = Patient(1, "haris", 22)
p2 = Patient(2, "Ammad", 25)
p1.add_history("Flu - January 2024")
p2.add_history("Checkup - March 2024")
appt = Appointment()
appt.schedule(p1, "Ali", "2025-11-07")
appt.schedule(p2, "Sara", "2025-11-08")
appt.view_all()
appt.cancel(p1)
appt.view_all()
```

output:

```
Added medical record for haris: Flu - January 2024
Added medical record for Ammad: Checkup - March 2024
Appointment booked for haris with Dr.Ali on 2025-11-07
Appointment booked for Ammad with Dr.Sara on 2025-11-08

--- Appointment List ---
haris → Dr.Ali on 2025-11-07
Ammad → Dr.Sara on 2025-11-08
Cancelled all appointments for haris.

--- Appointment List ---
Ammad → Dr.Sara on 2025-11-08
```

Question # 9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

```
9.py
lab 2 > 9.py > User
Windsurf: Refactor | Explain
class User:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = []
        self.posts = []
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_friend(self, friend):
        if friend not in self.friends:
            self.friends.append(friend)
            print(f"{self.username} added {friend.username} as a friend.")
        else:
            print(f"{friend.username} is already a friend.")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def create_post(self, text, private=False):
        self.posts.append({"text": text, "private": private})
        if private:
            print(f"{self.username} posted privately.")
        else:
            print(f"{self.username} posted publicly.")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def display_timeline(self):
        print(f"\n--- {self.username}'s Timeline ---")
        for p in self.posts:
            if not p["private"]:
                print(f"{self.username}: {p['text']}")
            else:
                print("This post is private.")
u1 = User("Haris", "haris@email.com")
u2 = User("Ammad", "ammad@email.com")
u1.add_friend(u2)
u1.create_post("Hello world!", private=False)
u1.create_post("Feeling sleepy...", private=True)
u1.display_timeline()
```

output:

```
Haris added Ammad as a friend.
Haris posted publicly.
Haris posted privately.

--- Haris's Timeline ---
Haris: Hello world!
This post is private.
```

Question # 10: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost

```python
class Room:
    def __init__(self, number, rtype, price):
        self.number = number
        self.rtype = rtype
        self.price = price
        self.available = True

class Booking:
    def __init__(self):
        self.records = []
    def reserve(self, room, nights):
        if room.available:
            room.available = False
            cost = room.price * nights
            self.records.append((room, nights, cost))
            print(f"Booked Room {room.number} for {nights} nights. Cost = Rs.{cost}")
        else:
            print("Room not available!")
    def checkout(self, room):
        room.available = True
        print(f"Checked out Room {room.number}")
    def view_bookings(self):
        if not self.records:
            print("No bookings made yet.")
        else:
            print("\n--- Booking Records ---")
            for r, n, c in self.records:
                print(f"Room {r.number} ({r.rtype}) - {n} nights - Rs.{c}")
# Example usage
r1 = Room(101, "Single", 4000)
r2 = Room(202, "Double", 6000)
b = Booking()
b.reserve(r1, 3)
b.reserve(r2, 2)
b.view_bookings()
b.checkout(r1)
```

output:

```
Booked Room 101 for 3 nights. Cost = Rs.12000
Booked Room 202 for 2 nights. Cost = Rs.12000

--- Booking Records ---
Room 101 (Single) - 3 nights - Rs.12000
Room 202 (Double) - 2 nights - Rs.12000
Checked out Room 101
```

Question # 11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

```python
class Course:
    def __init__(self, code, name, credits, capacity, schedule):
        self.code, self.name, self.credits, self.capacity, self.schedule = code, name, credits, capacity, schedule
        self.enrolled = []
    def register(self, student):
        if len(self.enrolled) < self.capacity:
            self.enrolled.append(student)
            print(f"{student.name} registered for {self.name}")
        else:
            print(f"{self.name} is full!")
class StudentReg:
    def __init__(self, sid, name):
        self.sid, self.name, self.courses, self.limit = sid, name, [], 18
    def add_course(self, course):
        if sum(c.credits for c in self.courses) + course.credits > self.limit:
            return print(f"{self.name} credit limit exceeded!")
        if any(c.schedule == course.schedule for c in self.courses):
            return print(f"Schedule conflict with {course.name}!")
        course.register(self)
        self.courses.append(course)
    def view_courses(self):
        if not self.courses:
            print(f"{self.name} has no courses.")
        else:
            print(f"\n{self.name}'s Courses:")
            for c in self.courses:
                print(f"{c.code} - {c.name} ({c.credits} credits, {c.schedule})")
```

```python
c1 = Course("CS101", "Intro to Programming", 3, 2, "Mon 10-12")
c2 = Course("CS102", "Data Structures", 3, 2, "Mon 10-12")
c3 = Course("MA101", "Calculus", 4, 3, "Tue 9-11")

s1 = StudentReg(1, "Haris")
s2 = StudentReg(2, "Ammad")

s1.add_course(c1)
s1.add_course(c2)
s1.add_course(c3)
s1.view_courses()
s2.add_course(c1)
s2.add_course(c1)
```

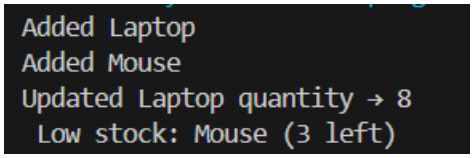
output:

```
Haris registered for Intro to Programming
Schedule conflict with Data Structures!
Haris registered for Calculus

Haris's Courses:
CS101 - Intro to Programming (3 credits, Mon 10-12)
MA101 - Calculus (4 credits, Tue 9-11)
Ammad registered for Intro to Programming
Schedule conflict with Intro to Programming!
```

Question # 12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

```
12.py
lab 2 > 12.py > ProductInv
Windsurf: Refactor | Explain
class ProductInv:
 Windsurf: Refactor | Explain | Generate Docstring | ×
 def __init__(self,pid,name,qty,price,supplier):
  self.pid=pid
  self.name=name
  self.qty=qty
  self.price=price
  self.supplier=supplier
Windsurf: Refactor | Explain
class Inventory:
 Windsurf: Refactor | Explain | Generate Docstring | ×
 def __init__(self):
  self.products=[]
 Windsurf: Refactor | Explain | Generate Docstring | ×
 def add_product(self,product):
  self.products.append(product)
  print(f"Added {product.name}")
 Windsurf: Refactor | Explain | Generate Docstring | ×
 def update_qty(self,pid,new_qty):
  for p in self.products:
   if p.pid==pid:
    p.qty=new_qty
    print(f"Updated {p.name} quantity → {new_qty}")
 Windsurf: Refactor | Explain | Generate Docstring | ×
 def low_stock_alert(self):
  for p in self.products:
   if p.qty<5:
    print(f" Low stock: {p.name} ({p.qty} left)")
p1=ProductInv(1,"Laptop",10,120000,"TechWorld")
p2=ProductInv(2,"Mouse",3,1500,"GadgetHub")
inv=Inventory()
inv.add_product(p1)
inv.add_product(p2)
inv.update_qty(1,8)
inv.low_stock_alert()
```

output:



Question # 13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

13.py

lab 2 > 13.py > Movie

```python
Windsurf: Refactor | Explain
class Movie:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,title,genre,duration):
        self.title=title
        self.genre=genre
        self.duration=duration
        self.showtimes=[]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_showtime(self,time):
        self.showtimes.append(time)
Windsurf: Refactor | Explain
class Theater:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,name,seats=50):
        self.name=name
        self.seats=seats
        self.booked=0
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def reserve(self,count,category="standard"):
        if self.booked+count<=self.seats:
            self.booked+=count
            price=500 if category=="standard" else 800
            total=price*count
            print(f"Booked {count} {category} tickets. Total = Rs.{total}")
        else:
            print("Not enough seats!")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def available_seats(self):
        print(f"Available seats: {self.seats-self.booked}")
m1=Movie("Inception","Sci-Fi",148)
m1.add_showtime("6:00 PM")
m1.add_showtime("9:00 PM")
t1=Theater("CineStar",100)
t1.reserve(4,"standard")
t1.reserve(2,"premium")
t1.available_seats()
```

output:

Question # 14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

```python
class Member:
    def __init__(self, mid, name, mtype):
        self.mid, self.name, self.mtype, self.attendance = mid, name, mtype, 0
    def mark_attendance(self):
        self.attendance += 1
class Trainer:
    def __init__(self, name, specialty):
        self.name, self.specialty = name, specialty
class Session:
    def __init__(self):
        self.sessions = []
    def schedule(self, member, trainer, time):
        self.sessions.append((member, trainer, time))
        print(f"Session: {member.name} with {trainer.name} at {time}")
# Example
m1 = Member(1, "Haris", "Premium")
t1 = Trainer("Ammad", "Fitness")
s = Session()
s.schedule(m1, t1, "10 AM")
m1.mark_attendance()
print(f"{m1.name}'s Attendance: {m1.attendance}")
```

output:

```
Session: Haris with Ammad at 10 AM
Haris's Attendance: 1
```

Question # 15: Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts

15.py

lab 2 > 15.py > transfer

```python
Windsurf: Refactor | Explain
class BankAccount:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, acc_no, holder, balance=0):
        self.acc_no, self.holder, self.balance = acc_no, holder, balance
    def deposit(self, amt): self.balance += amt
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def withdraw(self, amt):
        if amt <= self.balance: self.balance -= amt
        else: print("Insufficient funds!")
Windsurf: Refactor | Explain
class SavingsAccount(BankAccount):
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, acc_no, holder, balance=0, rate=0.03):
        super().__init__(acc_no, holder, balance)
        self.rate = rate
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add_interest(self):
        self.balance += self.balance * self.rate
        print(f"Interest added. New balance: {self.balance}")
Windsurf: Refactor | Explain
class CheckingAccount(BankAccount):
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self, acc_no, holder, balance=0, fee=50):
        super().__init__(acc_no, holder, balance)
        self.fee = fee
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def withdraw(self, amt):
        total = amt + self.fee
        if total <= self.balance:
            self.balance -= total
            print(f"Withdrew {amt}+Fee {self.fee}. Balance={self.balance}")
        else:
            print("Insufficient funds!")
```

```python
def transfer(frm, to, amt):
    if frm.balance >= amt:
        frm.balance -= amt
        to.balance += amt
        print(f"Transferred {amt} from {frm.holder} to {to.holder}")
    else:
        print("Transfer failed!")
# Example
s = SavingsAccount(101, "Haris", 1000)
c = CheckingAccount(102, "Ammad", 1500)
s.add_interest()
c.withdraw(300)
transfer(c, s, 500)
```

output:

```
Interest added. New balance: 1030.0
Withdrew 300+Fee 50. Balance=1150
Transferred 500 from Ammad to Haris
```

Question # 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

16.py

lab 2 > 16.py > Pizza

```python
Windsurf: Refactor | Explain
class Pizza:
    BASE={"small":300,"medium":500,"large":700}
    TOP=50
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,size,crust,toppings=None):
        self.size=size;self.crust=crust;self.toppings=toppings or []
    def price(self):return self.BASE[self.size]+len(self.toppings)*self.TOP
Windsurf: Refactor | Explain
class Order:
    def __init__(self):self.items=[]
    def add(self,pizza):self.items.append(pizza)
    def total(self):return sum(p.price()for p in self.items)
# Example
p1=Pizza("medium","thin",["cheese","mushroom"])
p2=Pizza("large","pan",["pepperoni"])
o=Order();o.add(p1);o.add(p2)
print("Total=",o.total())
```

output:

```
Total= 1350
```

Question # 17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

17.py

lab 2 > 17.py > Car

```python
Windsurf: Refactor | Explain
class Car:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,model,year,color,price):
        self.model=model;self.year=year;self.color=color;self.price=price
Windsurf: Refactor | Explain
class CarManufacturer:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,name):
        self.name=name;self.log=[]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def produce(self,model,year,color,price):
        c=Car(model,year,color,price);self.log.append(c);print(f"Produced {c.model} ({c.color},{c.year})")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def qc(self,car):
        print(f"QC passed for {car.model}")

# Example usage
m=CarManufacturer("Toyota")
m.produce("Corolla",2024,"White",4000000)
m.produce("Yaris",2023,"Blue",3600000)
m.qc(m.log[0])
```

output:

Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

18.py

lab 2 > 18.py > ...

```python
Windsurf: Refactor | Explain
class Teacher:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,name):
        self.name=name
Windsurf: Refactor | Explain
class Student:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,name):
        self.name=name;self.grades={};self.attendance=0
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def mark_attendance(self):
        self.attendance+=1
Windsurf: Refactor | Explain
class Classroom:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.students=[]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add(self,s):
        self.students.append(s)
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def grade(self,s,sub,g):
        s.grades[sub]=g
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def report(self):
        for s in self.students:
            print(f"{s.name}:Grades{s.grades},Attendance{s.attendance}")
```

```python
# Example usage
t=Teacher("Mr haris")
s1=Student("hamza")
s2=Student("huzaifa")
cls=Classroom()
cls.add(s1)
cls.add(s2)
s1.mark_attendance()
s1.mark_attendance()
s2.mark_attendance()
cls.grade(s1,"Math",90)
cls.grade(s2,"Science",85)
cls.report()
```

output:

```
hamza:Grades{'Math': 90},Attendance2
huzaifa:Grades{'Science': 85},Attendance1
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests

19.py

lab 2 > 19.py > ...

```
Windsurf: Refactor | Explain
class Flight:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,number,destination,capacity):
        self.number=number;self.destination=destination;self.capacity=capacity;self.passengers=[]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def book(self,passenger):
        if len(self.passengers)<self.capacity:
            self.passengers.append(passenger);print(f"{passenger.name} booked on flight {self.number}")
        else:
            print("Flight full!")
Windsurf: Refactor | Explain
class Passenger:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,name):
        self.name=name
# Example usage
f1=Flight("PK301","Karachi",2)
p1=Passenger("Haris")
p2=Passenger("Ammad")
p3=Passenger("hamza")
f1.book(p1)
f1.book(p2)
f1.book(p3)
```

output:

```
Haris booked on flight PK301
Ammad booked on flight PK301
Flight full!
```

Question # 20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

20.py

lab 2 > 20.py > SecurityCamera

```
Windsurf: Refactor | Explain
class SmartDevice:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,name):
        self.name=name;self.status="off"
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def turn_on(self):
        self.status="on";print(f"{self.name} is ON")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def turn_off(self):
        self.status="off";print(f"{self.name} is OFF")
class SmartLight(SmartDevice):pass
Windsurf: Refactor | Explain
class Thermostat(SmartDevice):
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def set_temp(self,temp):
        print(f"{self.name} temperature set to {temp}°C")
class SecurityCamera(SmartDevice):pass
Windsurf: Refactor | Explain
class SmartHome:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.devices=[]
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def add(self,d):
        self.devices.append(d)
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def activate_all(self):
        for d in self.devices:d.turn_on()
```

```
light=SmartLight("Bedroom Light")
ac=Thermostat("AC")
cam=SecurityCamera("Door Camera")

home=SmartHome()
home.add(light)
home.add(ac)
home.add(cam)

home.activate_all()
ac.set_temp(22)
```

output:

```
Bedroom Light is ON
AC is ON
Door Camera is ON
AC temperature set to 22°C
```

Lab 3

Numpy

Question 1: A weather station records temperatures (°C) for 7 days

lab 3 > 1.py > ...

```python
import numpy as np
np.set_printoptions(precision=4, suppress=True)
temps_list = [21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]
temps = np.array(temps_list)
avg_temp = temps.mean()
max_temp = temps.max()
min_temp = temps.min()
temp_range = max_temp - min_temp
print("Q1 - Temperatures array:", temps)
print("Average:", avg_temp)
print("Max:", max_temp)
print("Min:", min_temp)
print("Range:", temp_range)
print("-" * 60)
```

Output

```
Q1 - Temperatures array: [21.1 23.4 19.8 25.  26.5 24.1 22. ]
Average: 23.12857142857143
Max: 26.5
Min: 19.8
Range: 6.699999999999999
```

Question 2: A fruit store records daily sales of apples for 4 weeks

```python
import numpy as np
week1 = [12, 15, 14, 10, 9, 8, 7]
week2 = [11, 12, 13, 9, 5, 8, 6]
week3 = [7, 9, 12, 10, 11, 13, 12]
week4 = [10, 11, 12, 7, 8, 9, 11]

sales = np.array([week1, week2, week3, week4])  # shape (4,7)
weekly_totals = sales.sum(axis=1)               # sum per week
overall_max_index_flat = sales.argmax()         # flattened index
overall_max_week, overall_max_day = np.unravel_index(overall_max_index_flat, sales.shape)
best_week_index = weekly_totals.argmax()
print("Q2 - Sales array (4x7):\n", sales)
print("Weekly totals:", weekly_totals)
print("Overall highest sale at (week_index, day_index):", (overall_max_week, overall_max_day))
print("Week with most sales (0-based index):", best_week_index)
print("-" * 60)
```

Output

```
Q2 - Sales array (4x7):
 [ 7  9 12 10 11 13 12]
 [10 11 12  7  8  9 11]]
Weekly totals: [75 64 74 68]
Overall highest sale at (week_index, day_index): (0, 1)
Week with most sales (0-based index): 0
------------------------------------------------------------
```

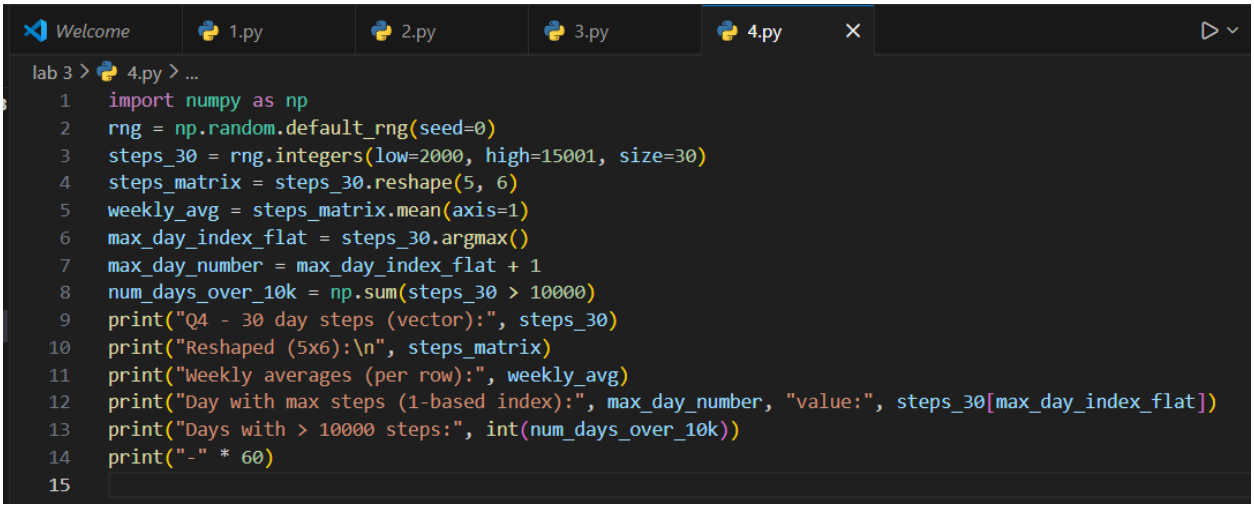
Question 3: Students received marks in a test

```python
import numpy as np
marks = np.array([56, 78, 45, 90, 88, 76, 59, 62])
marks_bonus = marks + 5  # broadcasting
count_above_60 = np.sum(marks_bonus > 60)
median_marks = np.median(marks_bonus)
std_marks = np.std(marks_bonus, ddof=0)
print("Q3 - Original marks:", marks)
print("Marks after +5 bonus:", marks_bonus)
print("Count > 60:", count_above_60)
print("Median:", median_marks)
print("Std dev:", std_marks)
print("-" * 60)
```

Output

```
Q3 - Original marks: [56 78 45 90 88 76 59 62]
Marks after +5 bonus: [61 83 50 95 93 81 64 67]
Count > 60: 7
Median: 74.0
Std dev: 15.105876340020794
```

Question 4: A smartwatch stores daily step-counts for a user for 30 days.



```python
import numpy as np
rng = np.random.default_rng(seed=0)
steps_30 = rng.integers(low=2000, high=15001, size=30)
steps_matrix = steps_30.reshape(5, 6)
weekly_avg = steps_matrix.mean(axis=1)
max_day_index_flat = steps_30.argmax()
max_day_number = max_day_index_flat + 1
num_days_over_10k = np.sum(steps_30 > 10000)
print("Q4 - 30 day steps (vector):", steps_30)
print("Reshaped (5x6):\n", steps_matrix)
print("Weekly averages (per row):", weekly_avg)
print("Day with max steps (1-based index):", max_day_number, "value:", steps_30[max_day_index_flat])
print("Days with > 10000 steps:", int(num_days_over_10k))
print("-" * 60)
```

Output

```
Q4 - 30 day steps (vector): [13058 10281  8645  5507  6002  2532  2978  2214  427
 [ 7124 13147  9206  2436 11944 11486]]
Weekly averages (per row): [ 7670.83333333  7725.33333333 10637.33333333  9067.16
666667
  9223.83333333]
Day with max steps (1-based index): 15 value: 14620
Days with > 10000 steps: 14
```

Question 5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day

5.py

lab 3 > 5.py > ...

```python
import numpy as np
rng = np.random.default_rng(seed=1)
hr = rng.integers(low=60, high=150, size=(10, 4))
daily_avg_per_patient = hr.mean(axis=1)
abnormal_mask = hr > 120
overall_max_hr = hr.max()
overall_min_hr = hr.min()
print("Q5 - Heart rates (10x4):\n", hr)
print("Daily average per patient:", np.round(daily_avg_per_patient, 2))
print("Abnormal readings (>120) mask:\n", abnormal_mask)
print("Overall max HR:", overall_max_hr)
print("Overall min HR:", overall_min_hr)
print("-" * 60)
```

Output

```
Q5 - Heart rates (10x4):
 [False False False False]
 [ True False False False]
 [ True False False False]]
Overall max HR: 147
Overall min HR: 62
------------------------------------------------------------
```

Question 6: Two branches of a store record monthly sales (in thousands)

5.py 6.py

lab 3 > 6.py > ...

```python
import numpy as np
A = np.array([122, 135, 150, 160, 155, 140])
B = np.array([110, 130, 148, 165, 158, 145])
diff = A - B
avg_A = A.mean()
avg_B = B.mean()
better_branch = "A" if A.sum() > B.sum() else ("B" if B.sum() > A.sum() else "Tie")
corr_coeff = np.corrcoef(A, B)[0, 1]
print("Q6 - Branch A:", A)
print("Branch B:", B)
print("Difference (A - B):", diff)
print("Avg A:", avg_A, "Avg B:", avg_B)
print("Better overall:", better_branch)
print("Correlation coefficient A vs B:", corr_coeff)
print("-" * 60)
```

Output

```
Q6 - Branch A: [122 135 150 160 155 140]
Branch B: [110 130 148 165 158 145]
Difference (A - B): [12  5  2 -5 -3 -5]
Avg A: 143.66666666666666 Avg B: 142.66666666666666
Better overall: A
Correlation coefficient A vs B: 0.9811677353198398
```

Question 7: Given a 4×4 grayscale image

7.py

lab 3 > 7.py > ...

```python
import numpy as np
img = np.array([
    [100, 120, 130, 90],
    [80 , 110, 140, 100],
    [70 , 90 , 120, 130],
    [110, 140, 150, 160]
], dtype=np.float64)
img_norm = img / 255.0
img_transformed = img * 1.2
img_transformed_clipped = np.clip(img_transformed, None, 255)
avg_before = img.mean()
avg_after = img_transformed_clipped.mean()
print("Q7 - Original image:\n", img)
print("Normalized (0-1):\n", np.round(img_norm, 4))
print("Transformed (x1.2) clipped to 255:\n", img_transformed_clipped)
print("Average brightness before:", avg_before)
print("Average brightness after:", avg_after)
print("-" * 60)
```

Output

```
Q7 - Original image:
 [[120. 144. 156. 108.]
 [ 96. 132. 168. 120.]
 [ 84. 108. 144. 156.]
 [132. 168. 180. 192.]]
Average brightness before: 115.0
Average brightness after: 138.0
```

Question 8: Dataset contains 100 samples with 4 features each.

```python
import numpy as np
rng = np.random.default_rng(seed=2)
X = rng.integers(1, 101, size=(100, 4)).astype(float)
means = X.mean(axis=0)
stds = X.std(axis=0, ddof=0)  # population std
X_scaled = (X - means) / stds
col_means = X_scaled.mean(axis=0)
col_vars = X_scaled.var(axis=0, ddof=0)
print("Q8 - X shape:", X.shape)
print("Column means (original):", np.round(means, 4))
print("Column stds (original):", np.round(stds, 4))
print("Column means after scaling (should be ~0):", np.round(col_means, 6))
print("Column variances after scaling (should be ~1):", np.round(col_vars, 6))
print("-" * 60)
```

Output

```
Q8 - X shape: (100, 4)
Column means (original): [51.3  53.18 52.3  48.28]
Column stds (original): [27.6632 30.3006 28.3335 28.3165]
Column means after scaling (should be ~0): [ 0.  0.  0. -0.]
Column variances after scaling (should be ~1): [1. 1. 1. 1.]
```

Question 9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

```python
import numpy as np
rng = np.random.default_rng(seed=3)
traffic = rng.integers(low=0, high=101, size=(24, 6))
hourly_avg = traffic.mean(axis=1)
sensor_totals = traffic.sum(axis=0)
busiest_sensor_index = sensor_totals.argmax()
traffic_capped = np.minimum(traffic, 50)
energy = (traffic_capped * 0.5).sum()
print("Q9 - Traffic shape:", traffic.shape)
print("Hourly averages (24):", np.round(hourly_avg, 2))
print("Busiest sensor (0-based index):", busiest_sensor_index, "total before cap:", sensor_totals[busiest_sensor_index])
print("Traffic after capping >50 -> 50 (sample first 5 hours):\n", traffic_capped[:5])
print("Energy (after cap) =", energy)
print("-" * 60)
```

Output

```
Q9 - Traffic shape: (24, 6)
 [[50  8 18 23 18 50]
 [50 50  3  9 33 43]
 [50 48 26 16 50 50]
 [ 3 11 45 39 50 50]
 [42 43 50 50 17 50]]
Energy (after cap) = 2745.0
```

Question 10: You are simulating forces in a mechanical system.

10.py

lab 3 > 10.py > ...

```python
import numpy as np
F = np.array([
    [5, 2, 1],
    [3, 1, 4],
    [6, 3, 2],
    [4, 2, 5],
    [7, 1, 3]
], dtype=float)
T = np.array([
    [2, 1, 0],
    [1, 3, 1],
    [0, 2, 2]
], dtype=float)
F_new = F.dot(T)  # shape (5,3)
magnitudes = np.linalg.norm(F_new, axis=1)
highest_idx = magnitudes.argmax()
eigvals, eigvecs = np.linalg.eig(T)
print("Q10 - Original F:\n", F)
print("Transformation T:\n", T)
print("Transformed forces F_new:\n", np.round(F_new, 4))
print("Magnitudes of each transformed force:", np.round(magnitudes, 4))
print("Component with highest transformed force (0-based index):", highest_idx, "magnitude:", magnitudes[highest_idx])
print("Eigenvalues of T:", np.round(eigvals, 6))
print("Eigenvectors of T (columns):\n", np.round(eigvecs, 6))
print("-" * 60)
```

Output

```
Q10 - Original F:
Eigenvalues of T: [4.302776 2.       0.697224]
Eigenvectors of T (columns):
 [[-0.311547  0.707107  0.386414]
 [-0.717422  0.       -0.50341 ]
 [-0.623093 -0.707107  0.772828]]
```

Lab 4

Predictions

Q1. Patient Stay Analysis (Healthcare)

You are given a dataset containing age, disease_type, and hospital_stay_days.
Write a Python program using **Seaborn** to:

- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    'age': [25, 40, 60, 35, 50, 70, 45, 55, 65, 30],
    'disease_type': ['Flu', 'Covid', 'Cancer', 'Flu', 'Covid',
                     'Cancer', 'Flu', 'Covid', 'Cancer', 'Flu'],
    'hospital_stay_days': [3, 10, 20, 4, 12, 25, 5, 9, 30, 2]
}

df = pd.DataFrame(data)
df

✓ 2.3s
```

Output:

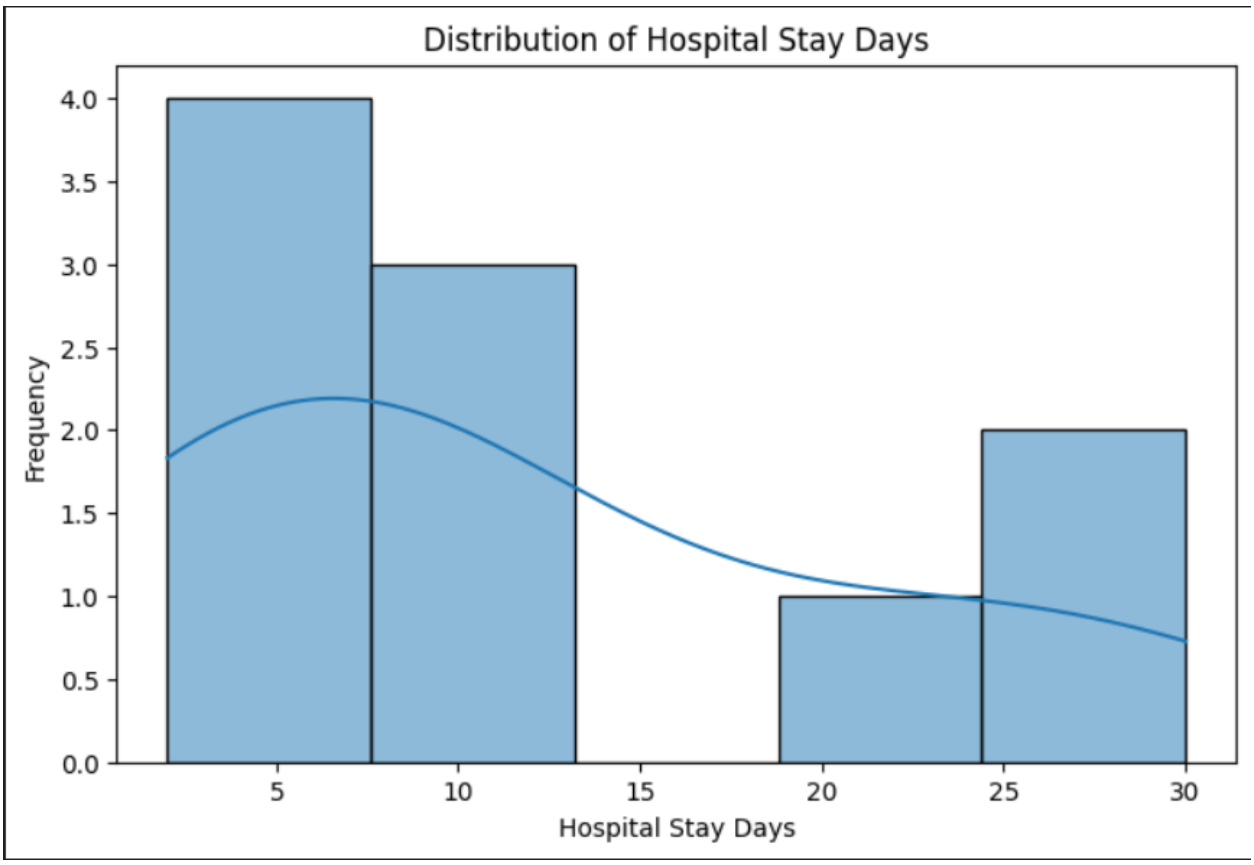
	age	disease_type	hospital_stay_days
0	25	Flu	3
1	40	Covid	10
2	60	Cancer	20
3	35	Flu	4
4	50	Covid	12
5	70	Cancer	25
6	45	Flu	5
7	55	Covid	9
8	65	Cancer	30
9	30	Flu	2

Code:

```
plt.figure(figsize=(8,5))
sns.histplot(df['hospital_stay_days'], kde=True)
plt.title("Distribution of Hospital Stay Days")
plt.xlabel("Hospital Stay Days")
plt.ylabel("Frequency")
plt.show()
```

✓ 0.2s

Output:

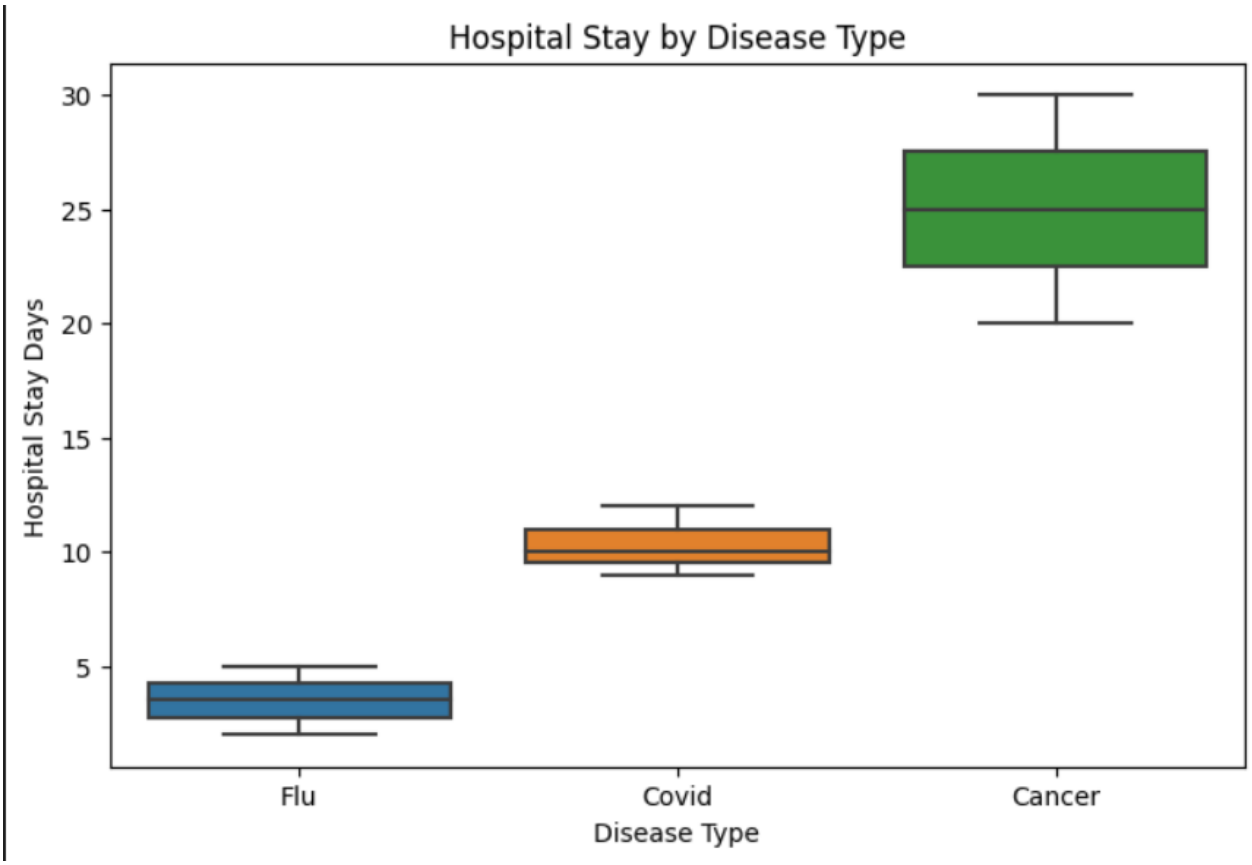


Code:

```
plt.figure(figsize=(8,5))
sns.boxplot(x='disease_type', y='hospital_stay_days', data=df)
plt.title("Hospital Stay by Disease Type")
plt.xlabel("Disease Type")
plt.ylabel("Hospital Stay Days")
plt.show()
```

✓ 0.1s

Output:




```
median_stay = df.groupby('disease_type')['hospital_stay_days'].median()
median_stay
```

✓ 0.0s

Output:

```
disease_type
Cancer    25.0
Covid     10.0
Flu        3.5
Name: hospital_stay_days, dtype: float64
```

Code:

```
highest_median_disease = median_stay.idxmax()
print("Disease with highest median hospital stay:", highest_median_disease)
```

✓ 0.0s

Output:

```
Disease with highest median hospital stay: Cancer
```

Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.
Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = {
    'region': ['North', 'South', 'East', 'West',
               'North', 'South', 'East', 'West',
               'North', 'South', 'East', 'West'],
    'monthly_sales': [20000, 15000, 18000, 22000,
                      25000, 14000, 17000, 26000,
                      30000, 16000, 19000, 28000]
}

df = pd.DataFrame(data)
df
```

✓ 1.3s

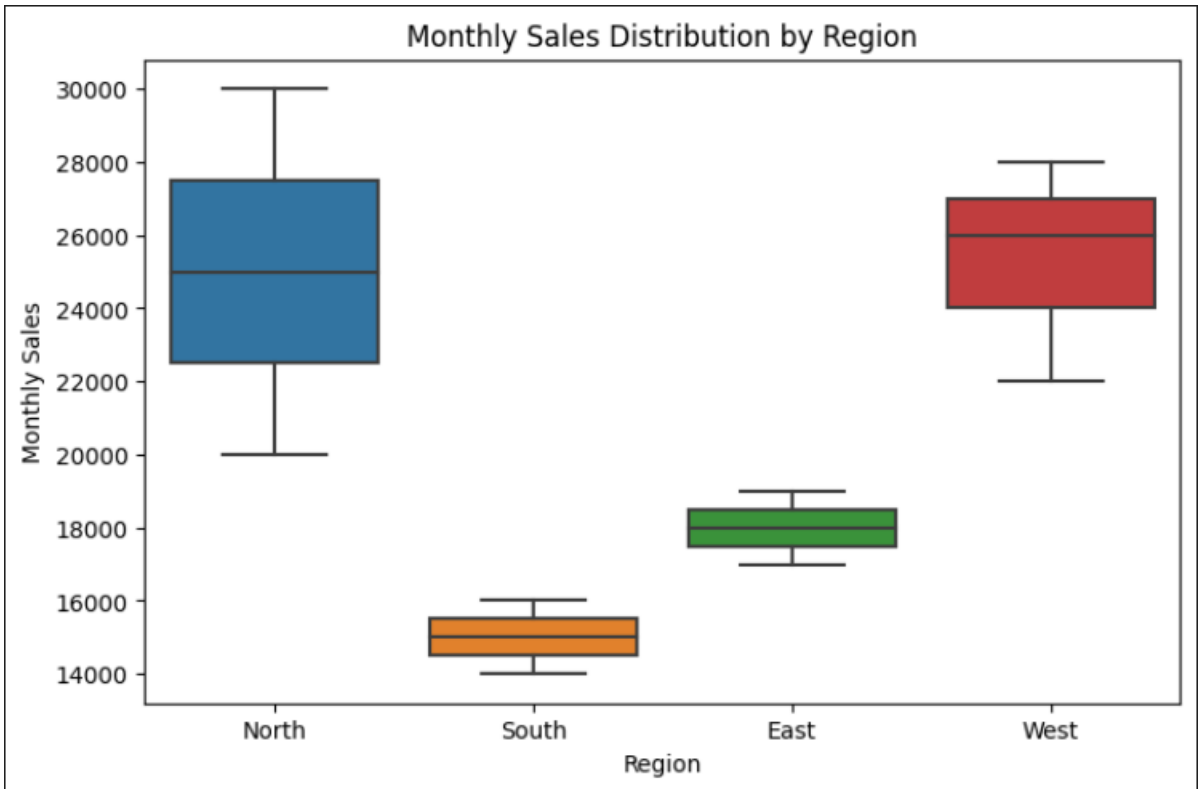
Output:

	region	monthly_sales
0	North	20000
1	South	15000
2	East	18000
3	West	22000
4	North	25000
5	South	14000
6	East	17000
7	West	26000
8	North	30000
9	South	16000
10	East	19000
11	West	28000

Code:

```
plt.figure(figsize=(8,5))
sns.boxplot(x='region', y='monthly_sales', data=df)
plt.title("Monthly Sales Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Monthly Sales")
plt.show()

✓ 0.6s
```

Output:



Code:

```
sales_variability = df.groupby('region')['monthly_sales'].std()
sales_variability
```

✓ 0.0s

Output:

```
...
```

region	monthly_sales
East	1000.000000
North	5000.000000
South	1000.000000
West	3055.050463

dtype: float64

Code:

```
high_variability_region = sales_variability.idxmax()
print("Region with highest sales variability:", high_variability_region)

✓ 0.0s
```

Output:

```
Region with highest sales variability: North
```

Q3. Student Performance Correlation (Education)

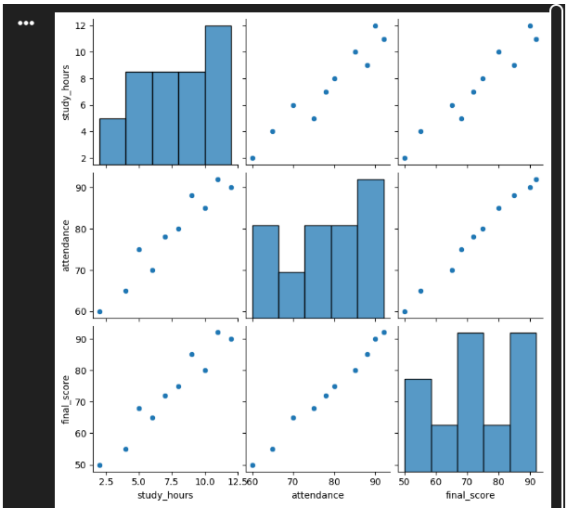
A dataset contains study_hours, attendance, and final_score .
Using Seaborn:

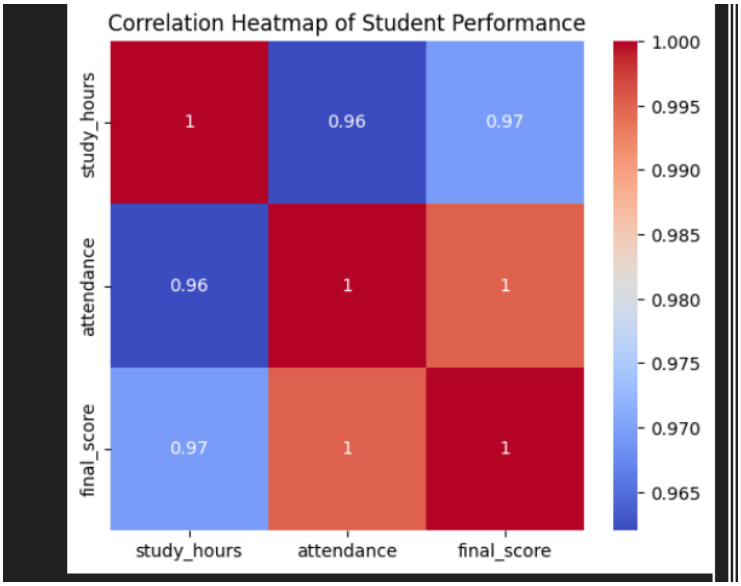
- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    'study_hours': [2, 4, 6, 8, 10, 12, 5, 7, 9, 11],
    'attendance': [60, 65, 70, 80, 85, 90, 75, 78, 88, 92],
    'final_score': [50, 55, 65, 75, 80, 90, 68, 72, 85, 92]
}

df = pd.DataFrame(data)
df
sns.pairplot(df)
plt.show()
plt.figure(figsize=(6,5))
corr = df.corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap of Student Performance")
plt.show()
corr['final_score']
```

Output:






```
study_hours    0.969399
attendance     0.995151
final_score    1.000000
Name: final_score, dtype: float64
```

Q4. Customer Segmentation Visualization

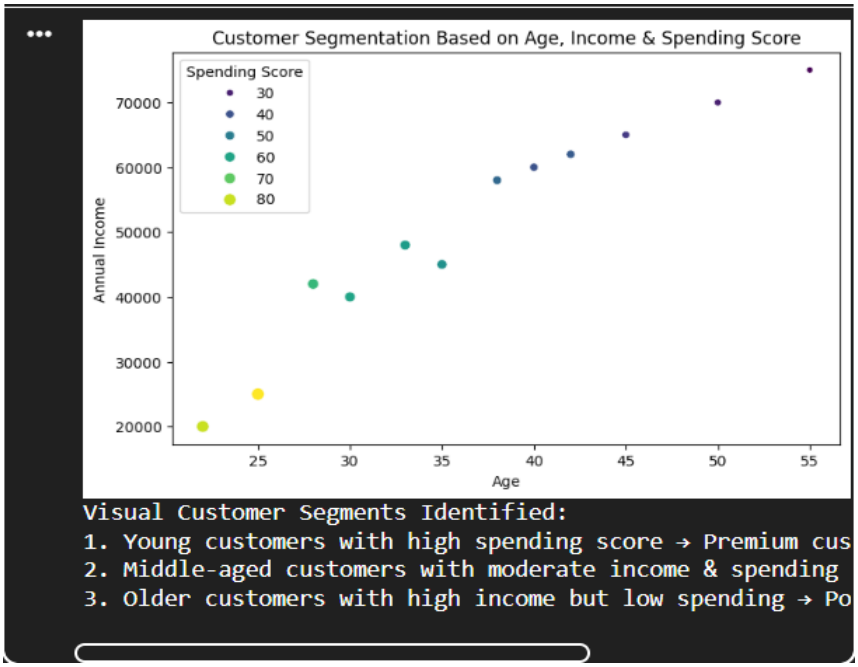
Customer data includes age, annual_income, and spending_score.
Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    'age': [22, 25, 30, 35, 40, 45, 28, 33, 38, 42, 50, 55],
    'annual_income': [20000, 25000, 40000, 45000, 60000, 65000,
                      42000, 48000, 58000, 62000, 70000, 75000],
    'spending_score': [80, 85, 60, 55, 40, 35, 65, 58, 45, 42, 30, 25]
}
df = pd.DataFrame(data)
plt.figure(figsize=(8,5))
sns.scatterplot(
    x='age',
    y='annual_income',
    hue='spending_score',
    palette='viridis',
    size='spending_score',
    data=df
)
plt.title("Customer Segmentation Based on Age, Income & Spending Score")
plt.xlabel("Age")
plt.ylabel("Annual Income")
plt.legend(title="Spending Score")
plt.show()
print("Visual Customer Segments Identified:")
print("1. Young customers with high spending score → Premium customers")
print("2. Middle-aged customers with moderate income & spending → Regular customers")
print("3. Older customers with high income but low spending → Potential targets for marketing")
```

Output:



Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.
Write code using Seaborn to:

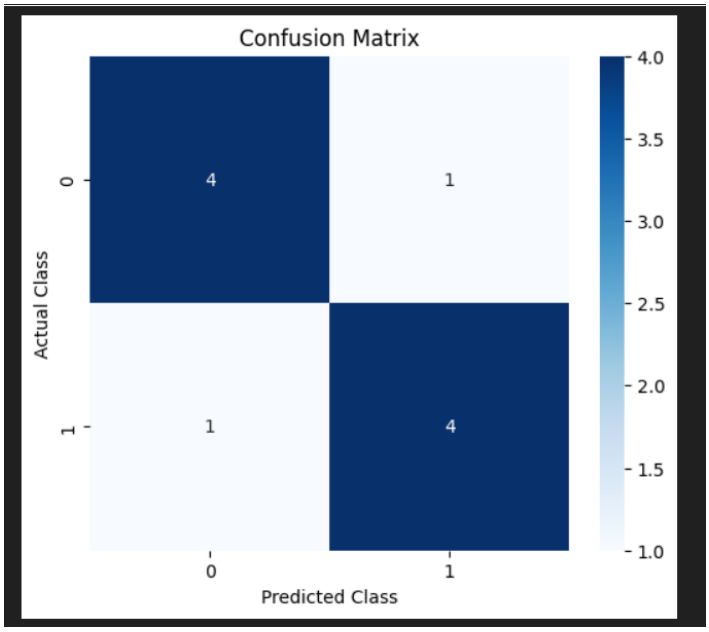
- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

Code:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
y_true = [0, 1, 0, 1, 1, 0, 1, 0, 0, 1]
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 1]
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    cbar=True
)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

✓ 0.8s

Output:



Q6. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.

Write a Scikit-learn program to:

- Encode categorical variables
- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

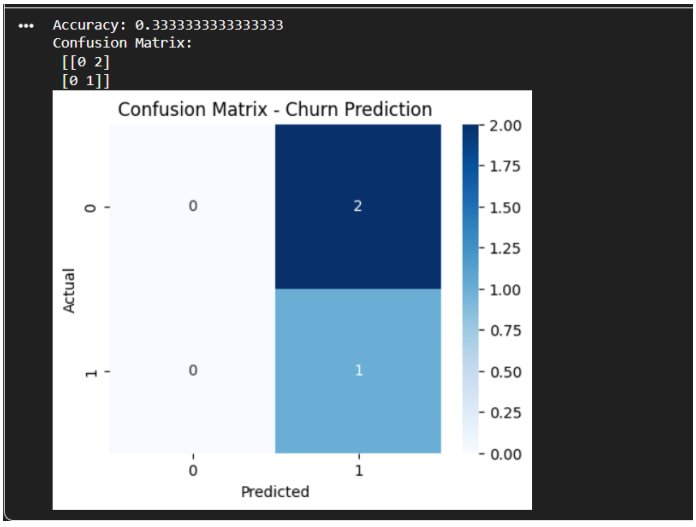
Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
data = {
    'tenure': [1, 5, 12, 24, 36, 6, 18, 30, 3, 48],
    'monthly_charges': [70, 80, 65, 90, 100, 75, 85, 95, 60, 110],
    'contract_type': ['Month-to-month', 'One year', 'Month-to-month',
                      'Two year', 'Two year', 'One year',
                      'Month-to-month', 'Two year', 'Month-to-month', 'Two year'],
    'churn': ['Yes', 'No', 'Yes', 'No', 'No', 'No', 'Yes', 'No', 'Yes', 'No']
}
df = pd.DataFrame(data)
df
```

```
le = LabelEncoder()
df['contract_type'] = le.fit_transform(df['contract_type'])
df['churn'] = le.fit_transform(df['churn'])
X = df[['tenure', 'monthly_charges', 'contract_type']]
y = df['churn']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = LogisticRegression()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
#  Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Churn Prediction")
plt.show()
```

✓ 1.4s

Output:



Q7. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.
Implement a regression model to:

- Split data into train/test sets
- Train Linear Regression
- Predict prices and calculate Mean Squared Error

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data = {
    'area': [800, 1000, 1200, 1500, 1800, 2000, 2200, 2500, 900, 1600],
    'bedrooms': [2, 2, 3, 3, 4, 4, 4, 5, 2, 3],
    'location_score': [6, 7, 8, 7, 9, 8, 9, 10, 6, 8],
    'price': [50000, 65000, 80000, 90000, 120000, 130000, 140000, 160000, 60000, 95000]
}

df = pd.DataFrame(data)
df

✓ 0.0s
```

Output:

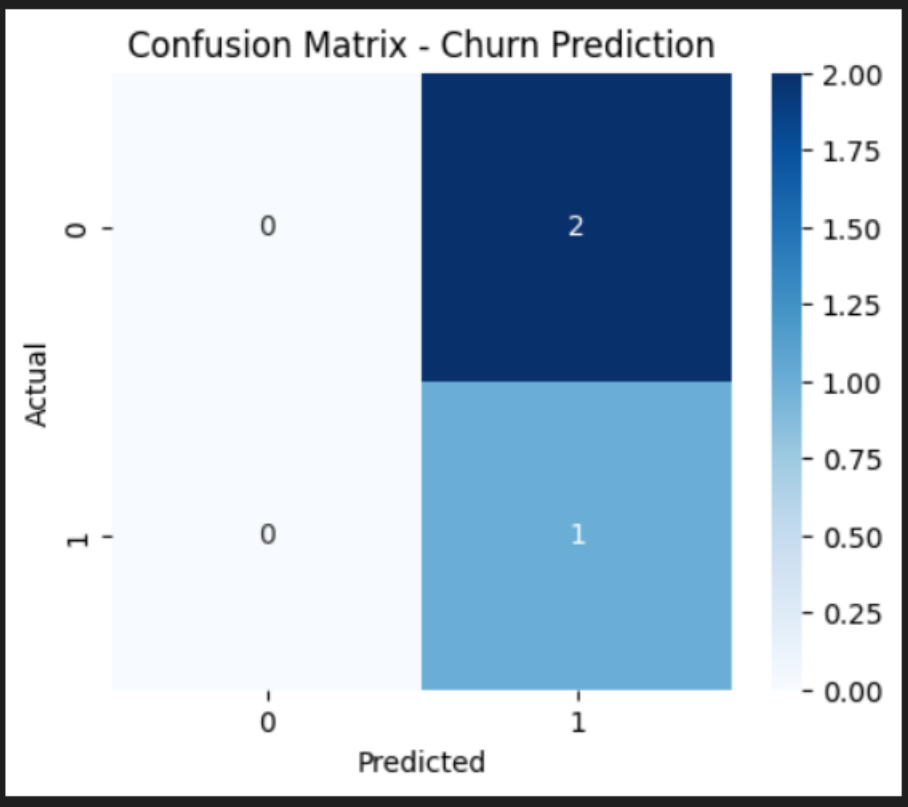
	area	bedrooms	location_score	price
0	800	2	6	50000
1	1000	2	7	65000
2	1200	3	8	80000
3	1500	3	7	90000
4	1800	4	9	120000
5	2000	4	8	130000
6	2200	4	9	140000
7	2500	5	10	160000
8	900	2	6	60000
9	1600	3	8	95000

Code:

```
X = df[['area', 'bedrooms', 'location_score']]
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# -------------------- Train Linear Regression Model --------------------
model = LinearRegression()
model.fit(X_train, y_train)

# -------------------- Predict House Prices --------------------
y_pred = model.predict(X_test)

# -------------------- Calculate Mean Squared Error --------------------
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
```

Output:

```
Accuracy: 0.3333333333333333
Confusion Matrix:
 [[0 2]
 [0 1]]
```



Q8. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.
Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

Code:

Output:

```
Mean Squared Error: 33816675.823235415
```

Q9. Email Spam Detection

You are given email text and spam labels.
Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

Code:

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
emails = [
    "Win money now",
    "Exclusive offer just for you",
    "Meeting scheduled tomorrow",
    "Project deadline extended",
    "Claim your free prize",
    "Let us discuss the report"
]
labels = [1, 1, 0, 0, 1, 0]   # 1 = Spam, 0 = Not Spam
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)
y = labels
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)
new_email = ["Win money now"]
new_email_vector = vectorizer.transform(new_email)
prediction = model.predict(new_email_vector)

if prediction[0] == 1:
    print("The email is SPAM")
else:
    print("The email is NOT SPAM")
```

0.0s Python

Output:

```
Model Accuracy: 0.0
The email is NOT SPAM
```

Q10. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.
Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
data = {
    'attendance': [60, 75, 90, 50, 85, 40, 70, 95, 55, 80],
    'mid_marks': [40, 65, 80, 35, 75, 30, 60, 85, 45, 70],
    'final_marks': [45, 70, 88, 40, 82, 35, 65, 92, 48, 78],
    'at_risk': ['Yes', 'No', 'No', 'Yes', 'No', 'Yes', 'No', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)
df

✓ 1.4s
```

Output:

	attendance	mid_marks	final_marks	at_risk
0	60	40	45	Yes
1	75	65	70	No
2	90	80	88	No
3	50	35	40	Yes
4	85	75	82	No
5	40	30	35	Yes
6	70	60	65	No
7	95	85	92	No
8	55	45	48	Yes
9	80	70	78	No

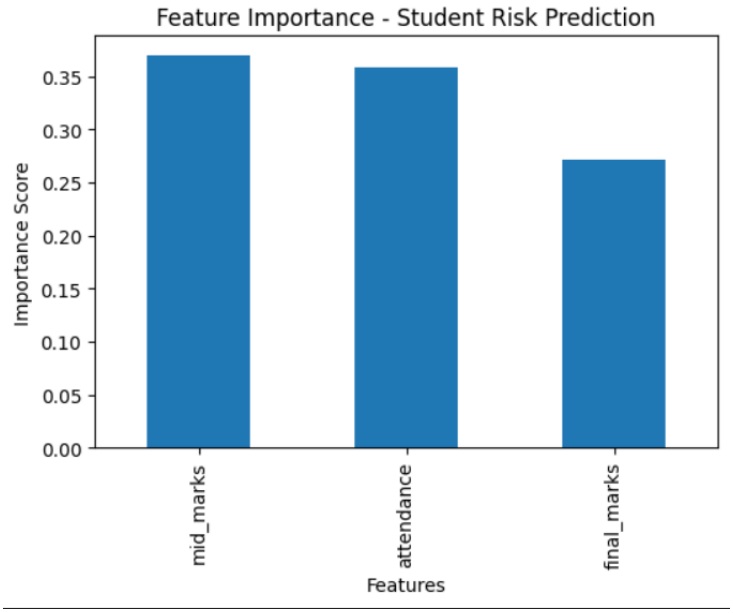
Code:

```
df['at_risk'] = df['at_risk'].map({'Yes': 1, 'No': 0})
X = df[['attendance', 'mid_marks', 'final_marks']]
y = df['at_risk']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)
feature_importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)
print("\nFeature Importance:")
print(feature_importance)
plt.figure(figsize=(6,4))
feature_importance.plot(kind='bar')
plt.title("Feature Importance - Student Risk Prediction")
plt.xlabel("Features")
plt.ylabel("Importance Score")
plt.show()
```

✓ 0.2s

Output:

```
...  Model Accuracy: 1.0

     Feature Importance:
     mid_marks      0.369565
     attendance     0.358696
     final_marks    0.271739
     dtype: float64
```



TensorFlow Library

What is TensorFlow?

TensorFlow is an **open-source library developed by Google** for machine learning (ML) and deep learning (DL). It is widely used to build and train neural networks for tasks like image recognition, natural language processing, and predictive modeling.

Key Features of TensorFlow:

- **Multi-platform:** Runs on CPU, GPU, mobile devices, and even browsers.
- **Flexible:** Supports high-level APIs like Keras for easy model building.
- **Scalable:** Can handle large datasets and distributed training.
- **Visualization:** Comes with TensorBoard for monitoring and visualizing model performance.

Where is TensorFlow used?

TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification
- Chatbots
- Medical analysis
- Recommendation systems (like Netflix)
- Self-driving cars

Code Examples

Basic Training

```python
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu', input_shape=(1,)),  # Specify input shape
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Convert to NumPy arrays and reshape
x = np.array([1, 2, 3, 4], dtype=np.float32).reshape(-1, 1)
y = np.array([2, 4, 6, 8], dtype=np.float32).reshape(-1, 1)

# Train the model
model.fit(x, y, epochs=50, verbose=0)  # verbose=0 to hide training output

# Predict - need to reshape the input for prediction too
print(model.predict(np.array([5]).reshape(-1, 1)))
```

Linear Regression

```python
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',          # Stochastic Gradient Descent optimizer
    loss='mean_squared_error' # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```